

AI HW4

Implementation Details

Part 1 : Game Environment Implementation

```
class BanditEnv:
    def __init__(self, k):
        self.k = k
        self.means = np.random.normal(0, 1, k)
        self.reset()

    def reset(self):
        self.means = np.random.normal(0, 1, self.k)
        self.action_history = []
        self.reward_history = []

    def step(self, action):
        reward = np.random.normal(self.means[action], 1)
        self.action_history.append(action)
        self.reward_history.append(reward)
        return reward

    def export_history(self):
        return self.action_history, self.reward_history
```

- reset()
Set the means for every arms and record the action and the corresponding reward.
- step(action)
Get the random reward from the normal distribution with selected action
- export_history()
return the history stat

Part 2 : Agent Implementation

```

class Agent:
    def __init__(self, k, epsilon):
        self.k = k
        self.epsilon = epsilon
        self.reset()

    def reset(self):
        self.Q = np.zeros(self.k) # Estimated rewards
        self.N = np.zeros(self.k) # Number of times each action was taken

    def select_action(self):
        if np.random.rand() < self.epsilon:
            return np.random.randint(self.k) # Exploration
        else:
            return np.argmax(self.Q) # Exploitation

    def update_q(self, action, reward):
        self.N[action] += 1
        self.Q[action] += (reward - self.Q[action]) / self.N[action]

```

- `select_action(self)`
select a random number from 0-1, if $< \epsilon$, do explore, if $> \epsilon$, just exploit.
- `update_q(self, action, reward)`
use sample average to update the reward after selection.
- `reset()`
set the Q to record the estimated reward, N for the number of times that the arm was taken.

Part 3 : Experiment

- Define the experiment part

```
def run_experiment(epsilon, runs=2000, steps=1000):
    avg_rewards = np.zeros(steps)
    optimal_actions = np.zeros(steps)

    for _ in range(runs):
        env = BanditEnv(10)
        agent = Agent(10, epsilon)
        optimal_action = np.argmax(env.means)

        for step in range(steps):
            action = agent.select_action()
            reward = env.step(action)
            agent.update_q(action, reward)

            avg_rewards[step] += reward
            if action == optimal_action:
                optimal_actions[step] += 1

    avg_rewards /= runs
    optimal_actions = (optimal_actions / runs) * 100
    return avg_rewards, optimal_actions
```

- Define the plotting part

```
def plot_results(results, steps, title_prefix, filename):
    plt.figure(figsize=(12, 5))

    # Plot Average Reward
    plt.subplot(1, 2, 1)
    for eps, (rewards, _) in results.items():
        plt.plot(steps, rewards, label=f' $\epsilon = \{eps\}$ ')
    plt.xlabel('Steps')
    plt.ylabel('Average Reward')
    plt.title(f'{title_prefix} - Average Reward')
    plt.legend()

    # Plot % Optimal Action
    plt.subplot(1, 2, 2)
    for eps, (_, optimal_actions) in results.items():
        plt.plot(steps, optimal_actions, label=f' $\epsilon = \{eps\}$ ')
    plt.xlabel('Steps')
    plt.ylabel('% Optimal Action')
    plt.title(f'{title_prefix} - % Optimal Action')
    plt.legend()

    plt.tight_layout()
    plt.savefig(filename)
    plt.close()
```

- Write main

```
if __name__ == "__main__":  
    # Stationary Experiment  
    epsilons = [0, 0.01, 0.1]  
    stationary_results = {}  
  
    for eps in epsilons:  
        rewards, optimal = run_experiment(eps)  
        stationary_results[eps] = (rewards, optimal)  
  
    plot_results(stationary_results, np.arange(1, 1001), 'Stationary', 'stationary_results.png')
```

Part 4 : Environment Implementation

- Modify BanditEnv to include a non-stationary reward distribution

```
def step(self, action):  
    if not self.stationary:  
        self.means += np.random.normal(0, 0.01, self.k)  
  
    reward = np.random.normal(self.means[action], 1)  
    self.action_history.append(action)  
    self.reward_history.append(reward)  
    return reward
```

Part 5 : Experiment

- Define the experiment function

```
def run_non_stationary_experiment(epsilon, runs=2000, steps=10000):
    avg_rewards = np.zeros(steps)
    optimal_actions = np.zeros(steps)

    for _ in range(runs):
        env = BanditEnv(10, stationary=False)
        agent = Agent(10, epsilon)
        for step in range(steps):
            optimal_action = np.argmax(env.means)
            action = agent.select_action()
            reward = env.step(action)
            agent.update_q(action, reward)

            avg_rewards[step] += reward
            if action == optimal_action:
                optimal_actions[step] += 1

    avg_rewards /= runs
    optimal_actions = (optimal_actions / runs) * 100
    return avg_rewards, optimal_actions
```

- write main

```
# Non-Stationary Experiment
non_stationary_results = {}
for eps in epsilons:
    rewards, optimal = run_non_stationary_experiment(eps)
    non_stationary_results[eps] = (rewards, optimal)

plot_results(non_stationary_results, np.arange(1, 10001), 'Non-Stationary', 'non_stationary_results.png')
```

Part 6 : Constant Step-Size Update Implementation

- Modify the agent

```
def update_q(self, action, reward):
    self.N[action] += 1
    if self.alpha is not None:
        # Constant step-size
        self.Q[action] += self.alpha * (reward - self.Q[action])
    else:
        # Sample-Average
        self.Q[action] += (reward - self.Q[action]) / self.N[action]
```

Part 7 : Experiment

- Define the experiment function

```
def run_constant_step_experiment(epsilon, alpha, runs=2000, steps=10000):
    avg_rewards = np.zeros(steps)
    optimal_actions = np.zeros(steps)

    for _ in range(runs):
        env = BanditEnv(10, stationary=False)
        agent = Agent(10, epsilon, alpha)
        for step in range(steps):
            optimal_action = np.argmax(env.means)
            action = agent.select_action()
            reward = env.step(action)
            agent.update_q(action, reward)

            avg_rewards[step] += reward
            if action == optimal_action:
                optimal_actions[step] += 1

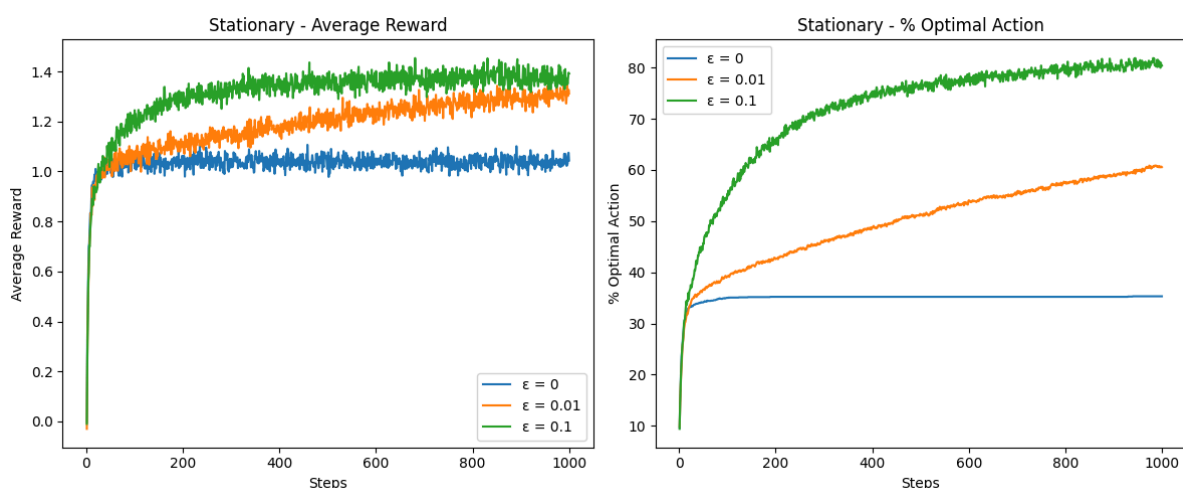
    avg_rewards /= runs
    optimal_actions = (optimal_actions / runs) * 100
    return avg_rewards, optimal_actions
```

- Write main

```
# Constant Step Experiment
constant_step_results = {}
alphas = [0.1, 0.01, 0.001]
for eps in epsilons:
    for alpha in alphas:
        rewards, optimal = run_constant_step_experiment(eps, alpha)
        constant_step_results[(eps, alpha)] = (rewards, optimal)
plot_results(constant_step_results, np.arange(1, 10001), 'Constant Step', 'constant_step_results.png')
```

Experiment Results

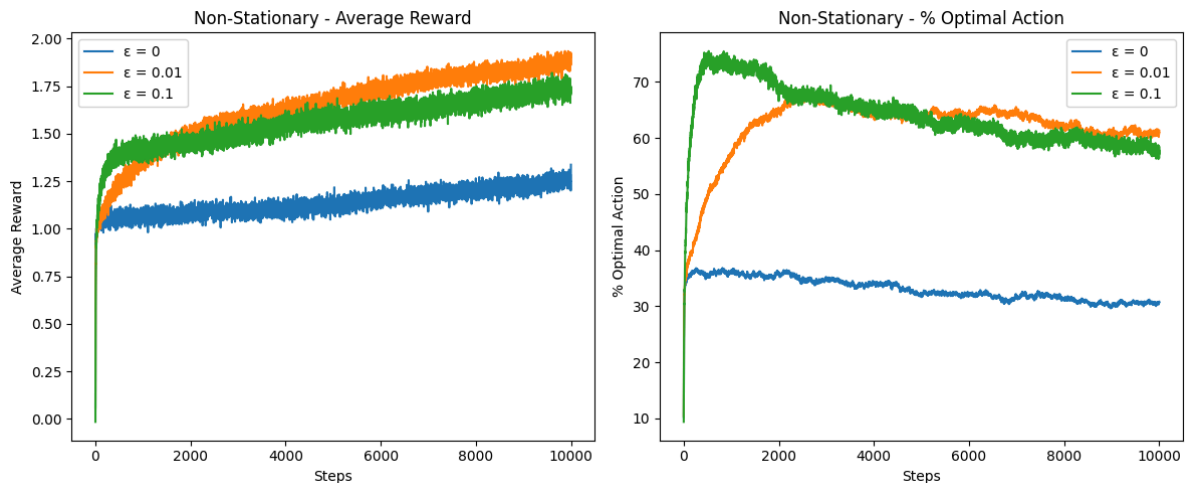
- **Part 3 : Stationary Environment (Sample-Average Update)**



- In stationary environments, higher exploration ($\epsilon=0.1$) still leads to faster convergence and higher long-term performance.

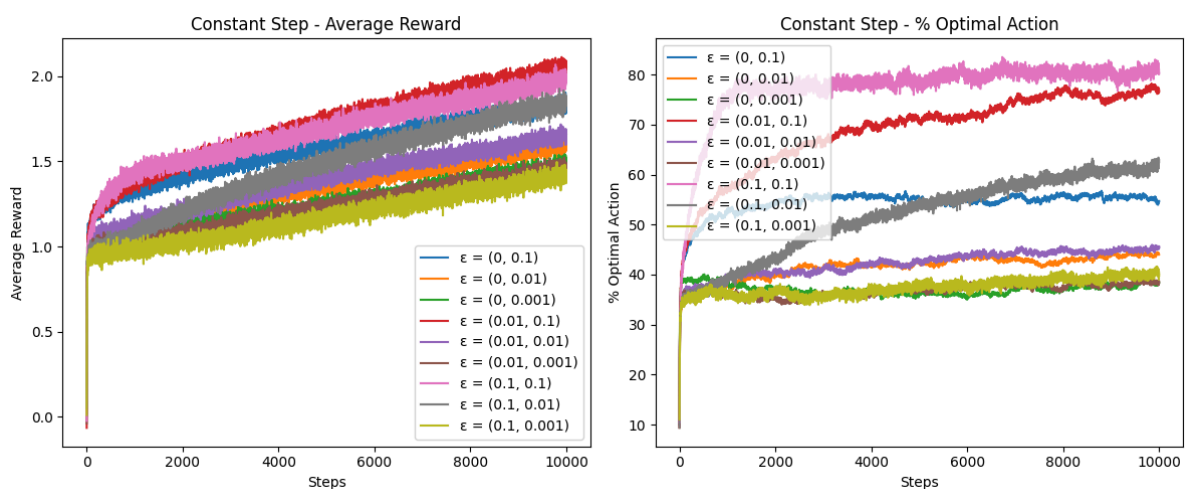
- $\epsilon=0$ performs poorly and plateaus quickly at a low optimal action selection rate ($\sim 30\%$).
- Exploration is important even in stable environments to find the best actions quickly.

• Part 5 : Non-Stationary Environment (Sample-Average Update)



- In non-stationary environments, higher ϵ ($\epsilon=0.1$) results in better performance since more exploration helps adapt to changing conditions.
- $\epsilon=0$ fails completely, getting stuck with outdated knowledge.

• Part 7 : Constant Step-Size (Non-Stationary Environment with α)



- Higher constant step sizes (α) lead to faster adaptation and higher average rewards, especially with larger ϵ values.
- $\epsilon=0.1$ combined with $\alpha=0.1$ performs best, achieving the highest average reward and optimal action selection.
- Without exploration ($\epsilon=0$), performance remains poor regardless of α .